Lecture 5

13 October 2026 22:54

We will now start to do self - attention in the model
till now our model Is not taking with its past tokens, which leads to evenly resulting in output

Mathematical trick for self-attention

1. We will train the model only till the 5 token, the 6 token, 7 token and similarly next token will be the future token which the model needs to predict

2. We have seen the concept before, this make the model actually predicting ethe token rather than just memorizing it and pasting it

There is a way we do this, similar the way we predict the data in Kalman filter, by first calculating average i.e. the low pass filter

What we will do, example there are 5 token, so the way this 5 token will be taking with the previous token is by averaging the previous token and coupling with the 5th token

V. IMP

LIKE.,
For the 5th token, first we will do the average till 4, same again the average till 3 and again till 2 and so on,
Although this seems to be a non-optimal way of combing the 5th token or we can say giving the information of the previous token, many data will get loss in this process, but we will bring it back later by some other mechanism

1.C.1

```
Torch.manual_seeds(1337)

B, T, C = 4, 8, 2
X = Torch.rand(B, T, C)
x.shape
```

Output:
torch.Size([4, 8, 2])

1.C.2

```
# We want mean of x[b, t} = mean_{i<=t} x[b, i]
xbow = torch.zeros((B, T, C))

for b in range(B):
   for t in range(T):
      xprev = x[b, :t+1]
      xbow[b, t] = torch.mean(xprev, 0)
```

Here the B, T, C tensor with sixe 4, 8, 2 is same as the one worth 4, 8, 65

Imagine it like a 8 rows with 2 columns each on 4 different pages like this:

```
      PAGE 1 (Batch 0)        PAGE 2 (Batch 1)
      C=0   C=1               C=0   C=1
T=0  [  0.41,  1.23 ]       [  0.09, -0.33 ]
T=1  [ -0.31,  0.82 ]       [ -1.20,  0.54 ]
T=2  [  0.92, -0.51 ]       [  0.44,  1.10 ]
T=3  [  0.11,  0.73 ]       [ -0.61, -0.29 ]
T=4  [ -0.44,  0.19 ]       [  1.33,  0.05 ]
T=5  [  1.02, -0.88 ]       [ -0.72,  0.91 ]
T=6  [ -0.19,  0.34 ]       [  0.28, -1.40 ]
T=7  [  0.56,  0.61 ]       [ -0.15,  0.67 ]

      PAGE 3 (Batch 2)        PAGE 4 (Batch 3)
      C=0   C=1               C=0   C=1
T=0  [  1.11, -0.42 ]       [ -0.88,  0.23 ]
T=1  [ -0.55,  0.18 ]       [  0.37, -0.91 ]
T=2  [  0.73,  0.99 ]       [ -0.14,  0.50 ]
...and so on                ...and so on
```

Why PyTorch prints it differently

When you print(x) in PyTorch, it shows the tensor as **nested brackets** which can look confusing:

text
tensor([[[page1_row1_col1, page1_row1_col2],

```
 [ page1_row2_col1, page1_row2_col2 ],
 ...],
[[ page2_row1_col1, page2_row1_col2 ],
 ...]])
```

The outermost bracket = batches (B), middle bracket = time steps (T), innermost bracket = features ([Equation]). Your "pages and rows and columns" mental model is exactly right — PyTorch just writes it as nested brackets instead of drawing it visually!

Side by Side Comparison:

	Real Bigram Model	This Demonstration
B	4 or 32 (batch size)	4
T	8 (block size)	8
C	**65** (vocab size)	**2** (simplified!)
Purpose	Actual language model	Just to demonstrate the math trick

V V IMP

What each position holds:

In the Real Model (4, 8, 65):
Each of the 32 token positions holds a **vector of 65 numbers** (one score per character in vocabulary):

text
Position [0][0]: [-2.45, 1.20, 0.88, ..., -0.33] ← 65 numbers
Position [0][1]: [1.10, -0.55, 2.31, ..., 0.71] ← 65 numbers
... and so on

In This Demo (4, 8, 2):
Each position holds only **2 numbers** (to keep things readable):

text
Position [0][0]: [0.41, 1.23] ← Just 2 numbers
Position [0][1]: [-0.31, 0.82] ← Just 2 numbers
... and so on

Step 1 :
xbow = torch.zeros((B, T, C))

Creates an **empty output tensor** of shape (4, 8, 2) filled with zeros. This is where we will store the averaged results. Think of it as a blank answer sheet.

for b in range(B): → Loop over 4 batches
for t in range(T): → Loop over 8 time positions
For every single position t in every batch b, we compute the average of all past tokens up to and including position t.

xprev = x[b, :t+1]
This slices the tensor to grab **all tokens from position 0 up to and including position t**:

Value of t	What x[b, :t+1] grabs	Shape
t = 0	Only token 0	(1, 2)
t = 1	Tokens 0 and 1	(2, 2)
t = 2	Tokens 0, 1, and 2	(3, 2)
t = 7	All 8 tokens	(8, 2)

Remember : V IMP -----NEED TO REMEMBER SO THAT HOW THE TENSOR'S ARE WORKING IS NEEDED

The inputs are pass in the form of batches with rows and columns, where each element in the each row is the token ID's of the input we are providing like this
Input: hello___
So the (B, T) will be just the input's token ID's [2, 4, 6, 8, , 9]

Where h = 2, e = 4, l = 6, l = 8, o = 9 and
When we get it in the (B, T, C) form,
EACH ROW represent's the probabilities of the alphabets
Like this

H ----> ['a' = 2.67, 'b' = 1.23, 'c' = 2.32, ...]
E -----> ['a' = -2.27, 'b' = -2.93, 'c' = 1.42, ...]
. . .
. . .
. . .
And so on.....

Understand it in this way [FINALLY]

1. The layout of x shape (B, T, C) — one batch page:

PAGE 1 (Batch 0) — One sequence like "hello..." from the dataset

```
              C=0     C=1    ...  C=64
T=0   'h':  [ 0.41,   1.23,  ...  -0.55 ]  ← 65 features of 'h'
T=1   'e':  [-0.31,   0.82,  ...   0.12 ]  ← 65 features of 'e'
T=2   'l':  [ 0.92,  -0.51,  ...   0.88 ]  ← 65 features of 'l'
T=3   'l':  [ 0.92,  -0.51,  ...   0.88 ]  ← 65 features of 'l'
T=4   'o':  [ 0.15,   0.74,  ...  -0.33 ]  ← 65 features of 'o'
...
T=7   ' ':  [-0.22,   0.44,  ...   0.61 ]  ← 65 features of ' '
```

V.V.Imp

1. Like batch are the different data set taken from the whole text randomly for traning
2. And, T = is the rows, which or most accurately, the TOKENS
3. And C = is the columns which when combine together, we can calculate the probabilty of the future most correct word

So:

- **Each ROW** (T dimension) = one **sequential character** from the input text.
- **Each COLUMN** (C dimension) = one **feature/dimension** of that character's embedding vector.

"If These are totally different sequential characters, why average their rows?"

When we compute xbow[b, 2] (the averaged context for position t=2, which is 'l'):

text
```
xprev = x[b, :3] =
  'h': [ 0.41,  1.23, ... ]
  'e': [-0.31,  0.82, ... ]
  'l': [ 0.92, -0.51, ... ]
mean along dim=0:
  [ (0.41-0.31+0.92)/3,  (1.23+0.82-0.51)/3, ... ]
= [ 0.34,  0.51, ... ]
```

The result [0.34, 0.51, ...] is **no longer purely 'l'**. It is a **blended memory vector** that says:
"I am the character 'l' at position 2, and I know that 'h' and 'e' came before me."

This blended vector carries **contextual history** into position 2. Without this, 'l' would have no idea what came before it!

2. Batches = Independent Separate Text Blocks

Each batch is a completely **independent random chunk** grabbed from somewhere in the Shakespeare dataset. They have **no connection to each other whatsoever**.

DATASET: "To be or not to be that is the question whether tis nobler..."

Batch 0: grabbed from position 1000 → "what light thr" → [45, 12, 1, 20, ...]
Batch 1: grabbed from position 5500 → "o be or not to" → [18, 1, 5, 14, ...]
Batch 2: grabbed from position 200 → "ll is well tha" → [11, 11, 1, 9, ...]
Batch 3: grabbed from position 8900 → "romeo wherefore" → [17, 18, 5, ...]

The reason we call them "rows" is just visual layout:

When printed as a 2D tensor (B=4, T=8), they happen to sit as 4 rows:

text
```
      ← T = 8 tokens →
Batch 0: [ 45, 12,  1, 20, 33,  4, 11,  5 ]  ← "what ligh"
Batch 1: [ 18,  1,  5, 14, 18,  2,  1, 20 ]  ← "o be or n"
Batch 2: [ 11, 11,  1,  9, 19,  1, 45, 5  ]  ← "ll is wel"
Batch 3: [ 17, 18,  5, 14,  1, 19,  1, 14 ]  ← "romeo whe"
```

But calling them "rows" is just a **visual convenience**. In reality they are:

Term	What it means
Rows	Just how the tensor looks when printed
Batches	Independent random text blocks from different parts of the dataset

Why independent? Each batch has its own separate context — Batch 1's 'o' has nothing to do with Batch 0's 'o'

----------understanding the code----------

Code:

```
xbow = torch.zeros((B, T, C))

for b in range(B):
   for t in range(T):
      xprev = x[b, :t+1]
      xbow[b, t] = torch.mean(xprev, 0)
```

Explanation:

Line 1: xbow = torch.zeros((B, T, C))

Creates a **blank output tensor** of shape (4, 8, 2) filled entirely with zeros.
Think of it as an **empty answer sheet** with the same shape as x. Every averaged result will be written into this tensor:

text
xbow initially:

Page 0: [[0,0], [0,0], [0,0], [0,0], [0,0], [0,0], [0,0], [0,0]]
Page 1: [[0,0], [0,0], [0,0], [0,0], [0,0], [0,0], [0,0], [0,0]]
Page 2: [[0,0], [0,0], [0,0], [0,0], [0,0], [0,0], [0,0], [0,0]]
Page 3: [[0,0], [0,0], [0,0], [0,0], [0,0], [0,0], [0,0], [0,0]]

VIMP

1. Where page 0, 1, 2, 3 aree separate pages like separate data-set randomly taken from the text
2. Each is having shape 1 cross 2 i.e. a, b but in real model it will be 1 cross 65 i.e. a, b, c ,.....z

Line 2: for b in range(B): → loops b = 0, 1, 2, 3

Loops over each of the **4 independent batch sequences** one at a time.
Each batch is processed completely separately — Batch 0's tokens never mix with Batch 1's tokens.

Line 3: for t in range(T): → loops t = 0, 1, 2, 3, 4, 5, 6, 7

For each batch, loops over every **time position** from 0 to 7.
This is asking: *"For token at position t, what is the average of all tokens up to and including t?"*

V V IMP:

Line 4: xprev = x[b, :t+1]

This is the **slicing step**. It grabs all tokens from position 0 up to and including position t for batch b:
Let's trace through Batch 0 step by step:

text

t=0: xprev = x[0, :1] → grabs only row 0 → shape (1, 2)
xprev = [[0.41, 1.23]]
HERE, T=0 is 1st token = 'h'

t=1: xprev = x[0, :2] → grabs rows 0 and 1 → shape (2, 2)
xprev = [[0.41, 1.23],
[-0.31, 0.82]]
HERE, T=1 is 2nd token = 'e'

t=2: xprev = x[0, :3] → grabs rows 0, 1, and 2 → shape (3, 2)
xprev = [[0.41, 1.23],
[-0.31, 0.82],
[0.92, -0.51]]
HERE, T=2 is 3rd token = 'l'

t=7: xprev = x[0, :8] → grabs all 8 rows → shape (8, 2)
HERE, T=7 is 8th token = '..something'

For t = 0 xprev = [[0.41, 1.23]] means -----> probability of 'a' coming and probability of 'b' coming ------> it is the row 0

For t = 1
xprev = [[0.41, 1.23],
[-0.31, 0.82]]] means -----> probability of 'a' coming and probability of 'b' coming ------> it is taking the row 0 and row

For t = 2
xprev = [[0.41, 1.23],
[-0.31, 0.82],
[0.92, -0.51]]
means -----> probability of 'a' coming and probability of 'b' coming ------> it is taking the row 0 and row and row 2

And so on,.....

For t = 3 xprev = [[0.41, 1.23]] means -----> probability of 'a' coming and probability of 'b' coming

And the reason why the tensor size is increasing cause we are considering the rows coming before it also.

Like this:

How Python Slicing Works:

array[start:end] means grab from index start up to but **NOT including** end.
When you write x[b, :t+1]:

- **No start** (:) means start from index 0.
- **End is t+1** means go up to but NOT including t+1, which means the last included index is t.

So :t+1 = **"from row 0 up to and including row t"**

Tracing through each value of t:

text
t=0: x[b, :0+1] = x[b, :1] → rows 0 to 0 → only Row 0
t=1: x[b, :1+1] = x[b, :2] → rows 0 to 1 → Row 0, Row 1
t=2: x[b, :2+1] = x[b, :3] → rows 0 to 2 → Row 0, Row 1, Row 2
t=3: x[b, :3+1] = x[b, :4] → rows 0 to 3 → Row 0, Row 1, Row 2, Row 3
...
t=7: x[b, :7+1] = x[b, :8] → rows 0 to 7 → All 8 rows

V IMP
Line 5: xbow[b, t] = torch.mean(xprev, 0)

torch.mean(xprev, 0) averages **along dimension 0** (collapses all the rows into one single row):

text
t=0: mean of 1 row:
= [0.41/1, 1.23/1]
= [0.41, 1.23] ← stored in xbow[0, 0]

t=1: mean of 2 rows:
= [(0.41 + -0.31)/2, (1.23 + 0.82)/2]
= [0.05, 1.025] ← stored in xbow[0, 1]

t=2: mean of 3 rows:
= [(0.41 + -0.31 + 0.92)/3, (1.23 + 0.82 + -0.51)/3]
= [0.34, 0.51] ← stored in xbow[0, 2]

The summary :

What xbow looks like after the loop for Batch 0:

text
xbow[0] after all 8 time steps:

T=0: [0.41, 1.23] ← average of just 'h'
T=1: [0.05, 1.02] ← average of 'h' + 'e' ---> while predicting the e, it has taken help of 'h', and so on down wards
T=2: [0.34, 0.51] ← average of 'h' + 'e' + 'l'
T=3: [0.28, 0.49] ← average of 'h' + 'e' + 'l' + 'l'
T=4: [0.26, 0.45] ← average of 'h' + 'e' + 'l' + 'l' + 'o'
T=5: [0.31, 0.33] ← average of first 6 tokens
T=6: [0.27, 0.34] ← average of first 7 tokens
T=7: [0.30, 0.40] ← average of all 8 tokens